

Prueba técnica

● Programa 1

Vista multi propósito (crear, ver, modificar) para prospectos de un nuevo crédito. El registro de cada prospecto es único por nombre. Los datos solicitados son: Nombre, teléfono, dirección, ciudad (catalogo), estado (catalogo), referencia (teléfono), email, equipo (catalogo), plazo (catalogo), enganche y total del equipo.

Para acceder a la vista, se requiere contar con el permiso número **6**, en otro caso se redirecciona a página principal.

Tiene una función que te reemplaza el catálogo de ciudad según el estado que se ha seleccionado. También valida todos los inputs antes de enviar la forma.

● Programa 2

Esta vista nos permite ver una entrada en bitácora con su desglose de factura listando sus mercancías. Además de permitir importar un archivo Excel que contenga mercancías para asignar a la entrada en bitácora solo si la forma no se encuentra bloqueada. Valida que el archivo subido no cuente con más de 512kb, que sea un documento de Excel con alguno de los mimes valido, y que las unidades y los productos descritos se encuentren registrados en las tablas de factura electrónica.

Cuenta también con la funcionalidad de generar la factura de la entrada en bitácora, este proceso valida si ya ha sido generada previamente. En caso de encontrar que ya se generó previamente registra en logs que es un duplicado y manda un correo de alerta. Caso contrario, registra en tabla de factura, en tabla de artículos y en tablas de traslados la información de la forma con sus calculos correspondientes.

Los campos que podemos ver en la forma son el folio de la bitácora, cliente, numero de unidad, permiso, recorrido (distancia), impuestos (IVA), retención, comentarios adicionales y observaciones.

Para acceder a la vista se requiere contar con alguno de los permisos **183** o **207**, caso contrario redirecciona a la página principal. También usa el permiso **209** para en caso de que se encuentre deshabilitada la forma poder activarla.

● Programa 3

Para poder ver el contenido de esta página es necesario tener el permiso **4** y tener asignada una división. En caso de no contar con división, hace redirección al index. En caso de no contar con el permiso, el programa no hace redirección, pero si muestra una leyenda de bienvenida y el recordatorio de siempre seguir los procedimientos.

Se asume que este programa es mainpage.php y que cada uno de los nombres listados (Laesquina, OneMobile, etc) son proveedores

La vista parece ser un panel/dashboard donde vemos información como el total de pagos, total de enganches, total cobrado, además de una tabla con información de ficheros de proveedores, unos grafico de barras ordenado por día de la semana/proveedor donde se muestra el monto total registrado en *credit* y las incidencias de la conciliación por cada uno de los proveedores. También cuenta con la funcionalidad para importar ficheros de proveedores, abriéndonos un modal donde se nos muestra una forma donde podemos seleccionar el proveedor y un documento. Dicha forma valida que el documento no pese más de 2mb. Al ser sometida es enviada con el parámetro `option` igual a `imp`.

Cuando se recibe el parámetro `option` pueden suceder 3 cosas.

- `con` : Muestra un mensaje con el número de pagos conciliados por proveedor

- `ftp` : Busca si hay registros en el día de documentos en el servidor para actualizar
- `imp` : Importa fichero e impacta las tablas correspondientes para cada proveedor

Esta vista tiene dos formas con el mismo ID

Preguntas

1. Menciona los métodos de consumo de un API, y proporciona algunos ejemplos.

Existen 5 métodos, cada uno con un propósito específico y siendo los dos primeros los principales

- **GET**: Se utiliza para consultar información, solo recibe data
- **POST**: Método para enviar datos, comúnmente utilizado para crear registros
- **PUT**: Útil para actualizar completamente un registro
- **PATCH**: Se llama a este método cuando se quiere actualizar solo una parte del registro
- **DELETE**: Eliminar el registro (Aunque normalmente solo se le hace un soft delete)

Un ejemplo de consumo de un api con jquery:

```
$.ajax({
  type: 'post',
  dataType: 'json',
  url: 'https://api.ejemplo.com/v1/crearRegistro',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': 'Bearer ALGUN_TOKEN_DE_ACCESO',
  },
  data: {
    nombre: 'John Doe',
    telefono: '9876543210',
  },
  success: (resp) => { ... },
  error: (err) => { ... },
})
```

Otro ejemplo usando javascript

```
const opciones = {
  method: 'put',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': 'Bearer ALGUN_TOKEN_DE_ACCESO',
  },
  body: JSON.stringify({
    nombre: 'John Deer',
    telefono: '9012345678',
  })
};

const consulta = fetch('https://api.ejemplo.com/v1/editarRegistro?rid=1', opciones)
  .then(resp => resp.json())
```

```
.then(data => { ... })  
.catch(err => { ... })
```

2. Que tipos de autorización hay, y proporciona un ejemplo.

A nivel usuario de sistema podemos tener autorización principalmente por dos medios; por roles y por permisos. Hay que mencionar que se complementan entre ellos. Además, se puede tener autorización a usar recursos a través de un token de acceso (JWT) con permisos o roles previamente asignados.

- Roles: Cada rol ya cuenta con una lista predeterminada de acciones (permisos) que tiene permitido realizar el usuario.
- Permisos: Se asignan manualmente a cada usuario, y pueden ser independiente al rol asignado. Cada permiso especifica una acción, una vista o un recurso al cual se puede acceder.

Existe también la autorización por características y por relación, pero considero se salen de la finalidad del cuestionario.

Un ejemplo de uso roles son los típicos de un sistema web; administrador, editor y usuario sin privilegios (usuario común). Donde el administrador tiene la capacidad de lectura, crear registros, editar registros (incluso sin ser de su autoría), eliminar recursos, además de poder brindar permisos adicionales a otros usuarios. Un editor tiene la capacidad de lectura y escritura sin la posibilidad de eliminar/cancelar registros o asignar permisos. Y por último el usuario común con la capacidad de solo lectura o acciones muy limitadas.

Un ejemplo de uso de roles en un sistema en laravel.

```
Route::middleware(['role:gerente'])->group(function() {  
    Route::resource('/admin', AdminController::class);  
})  
  
Route::middleware(['role:gerente|ejecutivo'])->group(function() {  
    Route::resource('/ventas', SalesDepartmentController::class);  
})
```

Un ejemplo de uso de permisos y roles en php.

```
define('PANEL_VENTAS', 123);  
  
if($_SESSION['rol'] == 'gerente' || in_array(PANEL_VENTAS, $_SESSION['permisos'])) {  
    verPanelVentas()  
}
```

3. Proporciona un ejemplo de un programa con laravel y VUE

```
<script setup>  
import { ref } from 'vue'  
  
const personaje = ref()  
const nombre = ref()  
const numero = ref()  
const altura = ref()
```

```

const peso = ref()
const imagen = ref()
const tipos = ref()

const buscarPersonaje = async () => {
  let pid = personaje.value

  if(!Number.isInteger(pid))
  {
    alert('Escribe un numero para continuar')
    return false
  }

  if(pid <= 0 || pid > 1025)
  {
    alert('Escribe un numero entre el 1 y 1025')
    return false
  }

  let options = {
    method: 'get',
    headers: {
      'Content-Type': 'application/json',
    },
  },
}

await fetch('https://pokeapi.co/api/v2/pokemon/'+pid, options)
  .then((resp) => resp.json())
  .then((data) => {
    nombre.value = data.name
    numero.value = String(data.id).padStart(4, '0')
    altura.value = data.height
    peso.value = data.weight
    imagen.value = data.sprites.front_default
    tipos.value = data.types.map((data) => data.type.name)
  })
  .catch((err) => {
    alert('Error al consultar datos')
    console.log(err)
  })
}
</script>

<template>
  <div class="bg-black">
    <div class="flex items-center justify-center h-screen">
      <div class="max-w-sm rounded-md shadow-md overflow-hidden bg-white">
        <div class="p-5">
          <h1 class="text-xl font-semibold mb-2">Busca tu pokemon</h1>
          <p class="text-gray-600 text-sm">
            Escribe en la caja de texto un numero entre el 1 y 1025, te mostraremos cual
            personaje corresponde.
          </p>
        </div>
      </div>
    </div>
  </div>

```

```

</p>

<div class="flex">
  <input type="number" id="personaje" class="w-full mt-4 px-4 py-2 border text-sm rounded-md shadow-xs" placeholder="Ej. 22" v-model="personaje">
</div>
<div class="flex">
  <button @click="buscarPersonaje" class="w-full mt-2 px-4 py-2 rounded-md text-white bg-blue-600 hover:bg-blue-800 transition">
    Buscar
  </button>
</div>
<div v-if="nombre" class="flex mt-4">
  <div class="w-1/2">
    <div class="flex items-center justify-center">
      
    </div>
  </div>
  <div class="w-1/2">
    <div class="text-sm">
      #{{ numero }}
    </div>
    <div class="text-sm font-bold">
      Nombre: <span class="font-normal capitalize">{{ nombre }}</span>
    </div>
    <div class="text-sm font-bold">
      Altura: <span class="font-normal">{{ altura }} pies</span>
    </div>
    <div class="text-sm font-bold">
      Peso: <span class="font-normal">{{ peso }} libras</span>
    </div>
    <div class="text-sm font-bold">
      Tipo: <span class="px-2 capitalize" v-for="(tipo, index) in tipos"
:key="index">{{ tipo }}</span>
    </div>
  </div>
</div>
</div>
</div>
</div>
</div>
</template>

```

 Nota adicional: Tratado de los programas

Para poder revisar el código de los tres programas enviados se utilizó Visual Studio Code, pero como no se lograban ver bien al copiar y pegar desde el correo enviado, se tuvo que trabajar un poco los documentos para mejor legibilidad.

1. Se reemplazaron los espacios en blanco con `ctrl + H`, en el input **buscar** se ingresó `^\s*$\n` y en el de **reemplazar** `\n`

2. Se agrego a la configuración del workspace **PHP Intelephense** como el formatear default
3. Se comentaron algunas y se agregó indentación en algunas líneas problemáticas para mejor visualización.